

Glossário de Termos — Aula 02

Herança, Polimorfismo, Composição e Classes Abstratas

Este glossário reúne os conceitos da Aula 02 — momento em que a Orientação a Objetos começa a mostrar todo o seu poder. Você aprenderá a reaproveitar código por herança, expressar relações entre objetos por composição, tratar diferentes tipos de forma uniforme com polimorfismo e modelar conceitos genéricos com classes abstratas. Use como referência durante os exercícios.

Herança — relação 'é-um'

Herança (Reaproveitamento de estrutura e comportamento)

Mecanismo que permite criar uma nova classe (subclasse) a partir de outra existente (superclasse), herdando seus atributos e métodos. Modela a relação 'é-um' (um Gato É-UM Animal, um Carro É-UM Veículo).

Analogia: É como herdar uma receita de família: você ganha pronta a base que sua avó fazia e pode acrescentar seus toques pessoais sem precisar começar do zero.

```
public class Animal {  
    String nome;  
    public void respirar() { System.out.println("Inspira/expira"); }  
}  
  
public class Cachorro extends Animal {  
    public void latir() { System.out.println("Au au!"); }  
}
```

Dica: Em Java, uma classe só pode herdar de UMA superclasse (herança simples). Para compartilhar comportamentos vindos de várias fontes, usamos interfaces — tema da próxima aula.

extends (Palavra-chave da herança)

Indica que uma classe herda de outra. Aparece na declaração logo após o nome da subclasse.

```
public class Carro extends Veiculo { ... }
```

super (Referência à superclasse)

Palavra-chave usada para acessar membros da superclasse a partir da subclasse. Muito comum em duas situações: chamar o construtor da superclasse (super(...)) e invocar um método sobrescrito da superclasse (super.metodo()).

```
public class Carro extends Veiculo {  
    private int portas;  
    public Carro(String modelo, int portas) {  
        super(modelo);    // chama construtor de Veiculo  
        this.portas = portas;  
    }  
}
```

Dica: super() precisa ser a PRIMEIRA instrução do construtor da subclasse. Se você não chamar explicitamente, o Java tenta chamar o super() sem argumentos — e se a superclasse não tiver esse construtor, o código não compila.

Sobrescrita (Override) (Redefinir comportamento herdado)

Quando uma subclasse redefine um método com a mesma assinatura herdado da superclasse. O Java passa a executar a versão da subclasse para objetos daquele tipo. A anotação `@Override` é fortemente recomendada para deixar a intenção explícita e permitir verificação pelo compilador.

```
public class Animal {  
    public void emitirSom() { System.out.println("som generico"); }  
}  
public class Gato extends Animal {  
    @Override  
    public void emitirSom() { System.out.println("miau!"); }  
}
```

Polimorfismo — muitos formatos

Polimorfismo (Mesmo nome, comportamentos diferentes)

Capacidade de tratar objetos de subclasses diferentes através do tipo da superclasse comum. Cada objeto responde à sua maneira à mesma chamada — quem decide qual método executar é o tipo REAL do objeto, não o tipo da variável.

Analogia: Pedir 'fale!' para diferentes animais: o cachorro late, o gato mia, a vaca muge. A mensagem é a mesma; a resposta depende de quem recebe.

```
Animal a1 = new Cachorro();
Animal a2 = new Gato();
a1.emitirSom(); // Au au!
a2.emitirSom(); // miau!
```

Dica: Esse é o tipo de polimorfismo mais usado: o de sobrescrita (override). Existe também o polimorfismo de sobrecarga (overload), em que vários métodos têm o mesmo nome mas assinaturas diferentes.

Upcasting (Atribuir subtipo a referência do supertipo)

Conversão implícita e segura: tratar um objeto de subclasse como se fosse da superclasse. É o que permite o polimorfismo funcionar.

```
Animal a = new Cachorro(); // upcasting – seguro
```

Downcasting (Forçar referência do supertipo a subtipo)

Conversão explícita e potencialmente arriscada: tratar uma referência de supertipo como se fosse de subtipo. Requer cast e pode lançar ClassCastException em tempo de execução se o objeto real não for do tipo esperado.

```
Animal a = new Cachorro();
Cachorro c = (Cachorro) a; // downcasting – funciona
c.latir();

Animal a2 = new Gato();
Cachorro c2 = (Cachorro) a2; // estoura em runtime!
```

Dica: Antes de fazer downcasting, use o operador instanceof para verificar o tipo real: if (a instanceof Cachorro) { ... }

Composição — relação 'tem-um'

Composição (Uma classe contém outra como atributo)

Forma de associar classes em que uma delas tem (HAS-A) outra como parte de seu estado. Em vez de herdar, a classe DELEGA responsabilidade ao objeto interno. É frequentemente mais flexível do que a herança.

Analogia: Um Carro TEM-UM Motor. Carro não é um motor, mas usa o motor para funcionar. Se o motor mudar (gasolina → elétrico), o Carro continua sendo Carro, mas delega o trabalho para o novo motor.

```
public class Motor {
    public void ligar() { System.out.println("Vrum!"); }
}

public class Carro {
    private Motor motor = new Motor(); // composicao
```

```
public void ligar() { motor.ligar(); }  
}
```

Dica: Regra de bolso famosa: 'prefira composição a herança'. Herança é forte e rígida (uma vez herdado, sempre herdado); composição é flexível (você pode trocar a parte interna sem mexer na classe).

Agregação (Forma fraca de composição)

Variante da composição em que o objeto contido pode existir independentemente do objeto que o contém. Composição forte: peças nascem e morrem juntas. Agregação: a peça pode ser compartilhada ou viver depois do todo.

Classes Abstratas — abstração parcial

Classe Abstrata (Conceito genérico, não instanciável)

Classe declarada com a palavra-chave `abstract` que NÃO pode ser instanciada com `new`. Serve como modelo base para subclasses concretas. Pode ter métodos implementados (com corpo) e métodos abstratos (sem corpo), além de atributos e construtores normais.

Analogia: É como o conceito de 'Veículo'. Você não compra 'um veículo' genérico — você compra um Carro, uma Moto, um Caminhão. Veículo descreve o que todos têm em comum, mas não existe Veículo solto na natureza.

```
public abstract class FormaGeometrica {  
    private String cor;  
    public FormaGeometrica(String cor) { this.cor = cor; }  
    public String getCor() { return cor; }  
  
    // metodo abstrato – toda subclasse DEVE implementar  
    public abstract double calcularArea();  
}
```

Dica: Tentar instanciar uma classe abstrata com `new` resulta em erro de compilação. A classe só ganha vida pelas subclasses concretas que a estendem.

Método Abstrato (Assinatura sem implementação)

Método declarado com a palavra-chave `abstract` dentro de uma classe abstrata. Não tem corpo (termina com ponto e vírgula). Toda subclasse concreta é obrigada a fornecer uma implementação para ele.

```
public class Circulo extends FormaGeometrica {  
    private double raio;  
    public Circulo(String cor, double raio) {  
        super(cor);  
        this.raio = raio;  
    }  
    @Override  
    public double calcularArea() {  
        return Math.PI * raio * raio;  
    }  
}
```

Dica: Se uma classe abstrata tem métodos abstratos não implementados, qualquer subclasse precisa decidir: ou implementa todos ou também se declara abstrata.

Outros conceitos da aula

`protected` (Modificador intermediário)

Modificador de acesso que libera o membro para a própria classe, classes do mesmo pacote e subclasses (mesmo em pacotes diferentes). Comum em superclasses que querem expor atributos para as filhas, mas não para o mundo.

`final` (em classe e método) (Trava herança e sobrescrita)

Classe declarada `final` não pode ser estendida (ex: `String`). Método declarado `final` não pode ser sobrescrito. Útil quando você quer garantir comportamento imutável dentro de uma hierarquia.

instanceof (Verificador de tipo em runtime)

Operador que testa se um objeto é instância de determinada classe (ou subclasse). Usado tipicamente antes de um downcasting para evitar ClassCastException.

```
if (animal instanceof Cachorro) {  
    Cachorro c = (Cachorro) animal;  
    c.latir();  
}
```

SENAC Entra21 — Módulo 07: Programação Orientada a Objetos — Material de apoio ao aluno